Project 2: Motion Planning

Shambhavi Singh

I. Introduction

Motion planning is the problem of computing a feasible trajectory from a start configuration to a goal configuration while avoiding obstacles. In this project, the robot operates in a continuous three-dimensional Euclidean space with a rectangular boundary and rectangular obstacle blocks. The goal is to design one planner that works for both static-goal environments and dynamic-goal environments, where the target appears at eight goal positions sequentially.

This problem can be viewed as a deterministic shortest path (DSP) problem, where configurations are states, feasible motions are edges, and edge costs represent motion length. Since explicitly constructing a dense graph in continuous 3-D space is expensive, this project uses a sampling-based RRT* planner. The planner incrementally builds a tree of collision-free configurations and improves path quality through best-parent selection and rewiring.

The project addresses three tasks: continuous collision checking between line segments and axis-aligned bounding boxes (AABBs), planning to static goals in environments E1–E5, and replanning to dynamic goals in environments E6–E7. For dynamic targets, the planner reuses the previously sampled tree across planning queries so that later goals can connect more quickly to already explored regions of free space. Performance is evaluated using path visualizations, plane projections, path length, explored nodes, iteration count, and computation time.

II. Problem Statement

The robot is modeled as a point moving in continuous three-dimensional Euclidean space. A configuration is

$$x = [x, y, z]^T \in \mathbb{R}^3.$$

The environment consists of a rectangular boundary

$$\mathcal{B} \subset \mathbb{R}^3$$

and a finite set of axis-aligned rectangular obstacle blocks

$$\mathcal{O}_1, \mathcal{O}_2, \ldots, \mathcal{O}_m \subset \mathbb{R}^3.$$

Each obstacle is represented by its lower and upper corners,

$$\mathcal{O}_j = [x^j_{\min}, x^j_{\max}] \times [y^j_{\min}, y^j_{\max}] \times [z^j_{\min}, z^j_{\max}].$$

The collision-free configuration space is

$$\mathcal{X}_{free} = \mathcal{B} \setminus \bigcup_{j=1}^{m} \mathcal{O}_j.$$

Given a start configuration $x_s \in \mathcal{X}_{free}$ and a goal configuration $x_g \in \mathcal{X}_{free}$, the objective is to find a continuous path

$$\sigma : [0, 1] \to \mathcal{X}_{free}$$

such that

$$\sigma(0) = x_s, \qquad \sigma(1) = x_g, \qquad \sigma(\alpha) \in \mathcal{X}_{free} \quad \forall \alpha \in [0, 1].$$

The path cost is its Euclidean arc length,

$$J(\sigma) = \int_0^1 \left\| \frac{d\sigma(\alpha)}{d\alpha} \right\|_2 d\alpha.$$

The optimal motion-planning problem is therefore

$$\sigma^* \in \arg\min_{\sigma} J(\sigma)$$

subject to the boundary, obstacle-avoidance, start, and goal constraints.

This problem can also be interpreted as a deterministic shortest path problem over a graph embedded in free space. Let

$$G = (V, E),$$

where each vertex $i \in V$ corresponds to a collision-free configuration $x_i \in \mathcal{X}_{free}$. An edge $(i, j) \in E$ is feasible only if the straight-line segment between the configurations is collision-free:

$$[x_i, x_j] \subset \mathcal{X}_{free}.$$

The edge cost is

$$c_{ij} = \begin{cases} \|x_j - x_i\|_2, & \text{if } [x_i, x_j] \subset \mathcal{X}_{free}, \\ \infty, & \text{otherwise.} \end{cases}$$

Let s denote the start node and τ denote the goal node. A feasible path is a sequence

$$i_{1:q} = (i_1, i_2, \ldots, i_q),$$

where

$$i_1 = s, \qquad i_q = \tau, \qquad c_{i_k i_{k+1}} < \infty \quad \text{for } k = 1, \ldots, q-1.$$

The deterministic shortest path objective is

$$\text{dist}(s, \tau) = \min_{i_{1:q}} \sum_{k=1}^{q-1} c_{i_k i_{k+1}}.$$

In Part 1, the planner must determine whether a candidate straight-line edge intersects any obstacle block. In Part 2, it must compute collision-free paths from a fixed start to a fixed goal in environments E1–E5. In Part 3, the obstacle set and start remain fixed, but the goal changes over time through

$$x_{g,1}, x_{g,2}, \ldots, x_{g,8}.$$

For each revealed goal $x_{g,r}$, the planner must compute a path from x_s to the current goal before the target moves to the next position. Thus, the dynamic-goal setting is a sequence of shortest path queries

$$(s, \tau_1), (s, \tau_2), \ldots, (s, \tau_8)$$

over the same collision-free configuration space.

III. Technical Approach

A. Part 1: Collision Checking Between Line Segments and AABBs

The first component is continuous collision checking between a candidate line segment and the axis-aligned rectangular obstacle blocks. Each obstacle is represented as an AABB,

$$\mathcal{O} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}].$$

A candidate edge between configurations $p_0, p_1 \in \mathbb{R}^3$ is parameterized as

$$p(t) = p_0 + t(p_1 - p_0), \qquad t \in [0, 1].$$

The edge is valid only if the full segment remains outside every obstacle.

I implemented a slab-based segment–AABB intersection test. For each coordinate axis $k \in \{x, y, z\}$, let

$$d_k = p_{1,k} - p_{0,k}.$$

The algorithm maintains an interval $[t_{\text{enter}}, t_{\text{exit}}]$, initialized as $[0, 1]$, representing the portion of the segment that could lie inside the box. If $|d_k|$ is close to zero, the segment is approximately parallel to the slab. If $p_{0,k}$ lies outside the slab bounds, the segment cannot intersect the box; otherwise, that coordinate does not shrink the interval.

If $d_k \neq 0$, the entry and exit parameters for the slab are

$$t_1 = \frac{b_{\min,k} - p_{0,k}}{d_k}, \qquad t_2 = \frac{b_{\max,k} - p_{0,k}}{d_k}.$$

After swapping them if necessary so that $t_1 \leq t_2$, the global interval is updated as

$$t_{\text{enter}} = \max(t_{\text{enter}}, t_1), \qquad t_{\text{exit}} = \min(t_{\text{exit}}, t_2).$$

If

$$t_{\text{enter}} > t_{\text{exit}},$$

the segment does not intersect the obstacle. If the interval remains nonempty after all three axes are checked, the segment intersects the AABB and is marked as a collision.

This test is applied to every candidate edge before adding it to the planner tree. Checking only sampled points is insufficient because two endpoints can be collision-free while the segment between them passes through an obstacle. If there are m obstacle blocks, one edge check has worst-case cost $O(m)$.

Algorithm 1 Segment–AABB Collision Check

for each obstacle block $\mathcal{O}$ **do**
  Initialize $t_{\text{enter}} = 0$, $t_{\text{exit}} = 1$
  Set `intersects = true`
  for each coordinate axis $k \in \{x, y, z\}$ **do**
    Compute $d_k = p_{1,k} - p_{0,k}$
    if $|d_k|$ is close to zero **then**
      if $p_{0,k}$ is outside the slab bounds **then**
        Set `intersects = false`
        Break
      end if
    else
      Compute slab entry and exit times t_1, t_2
      Swap t_1, t_2 if needed so that $t_1 \leq t_2$
      Update $t_{\text{enter}} = \max(t_{\text{enter}}, t_1)$
      Update $t_{\text{exit}} = \min(t_{\text{exit}}, t_2)$
      if $t_{\text{enter}} > t_{\text{exit}}$ **then**
        Set `intersects = false`
        Break
      end if
    end if
  end for
  if `intersects` **then**
    Return collision
  end if
end for
Return collision-free

B. Part 2: Static-Goal Planning with RRT*

For environments E1–E5, I implemented a sampling-based RRT* planner. The planner builds a tree of collision-free configurations rooted at the start. Each node stores a 3-D coordinate, a parent pointer, and a cost-to-come value g. The start node is initialized with

$$g(x_s) = 0,$$

while newly generated nodes are initialized with

$$g(x_{\text{new}}) = \infty$$

until a valid parent is selected. Once a collision-free parent is chosen, the new node receives a finite cost equal to the parent cost plus the Euclidean edge length.

The implementation maintains

`nodes`, `coordinates`, `tree`.

The list `nodes` stores node objects with coordinates, parents, and g-values. The array `coordinates` stores the same coordinates in matrix form. The `tree` is a SciPy `cKDTree` used for nearest-neighbor and radius-neighborhood queries. This avoids scanning all nodes during every nearest and near-node computation.

At each iteration, the planner samples a point in the bounded 3-D environment. With probability p_{goal}, the sampled point

is the goal; otherwise, it is drawn uniformly from the map boundary. For the final visualizations, I used

$$\epsilon = 0.5, \qquad p_{\text{goal}} = 0.1, \qquad N_{\max} = 100000.$$

Here, ϵ is the steering distance, p_{goal} is the goal sampling probability, and $N_{\max}$ is the maximum iteration count.

a) Extend Step: Given a sample x_{rand}, the planner finds the nearest node in the tree:

$$x_{\text{near}} = \arg\min_{x_i \in V} \|x_i - x_{\text{rand}}\|_2.$$

It then extends from x_{near} toward x_{rand}. If the sample is farther than ϵ, the new node is placed one step of length ϵ in that direction:

$$x_{\text{new}} = x_{\text{near}} + \epsilon \frac{x_{\text{rand}} - x_{\text{near}}}{\|x_{\text{rand}} - x_{\text{near}}\|_2}.$$

If x_{rand} is already within distance ϵ, it is used directly. This controls the resolution of exploration: smaller ϵ values give finer motion around obstacles, while larger values expand the tree faster.

Before insertion, the edge from x_{near} to x_{new} is checked using the continuous collision checker. If the edge intersects an obstacle, the sample is rejected. Thus, every edge added to the tree is collision-free in continuous 3-D space.

b) Best-Parent Selection: In vanilla RRT, x_{new} is usually connected directly to x_{near}. RRT* instead checks nearby nodes and selects the parent that gives the lowest cost-to-come. The neighborhood is

$$\mathcal{N}(x_{\text{new}}) = \{x_i \in V \mid \|x_i - x_{\text{new}}\|_2 \le r\},$$

where

$$r = 2\left(1 + \frac{1}{d}\right)^{1/d} \left(\frac{\text{Vol}(\mathcal{X}_{free})}{\text{Vol}(\mathcal{B}_d)}\right)^{1/d} \left(\frac{\log |V|}{|V|}\right)^{1/d}.$$

Here $d = 3$, $\text{Vol}(\mathcal{B}_d) = \frac{4}{3}\pi$, and the free-space volume is approximated as the boundary volume minus total obstacle volume. Among the nearby nodes, the planner chooses the collision-free parent minimizing

$$g(x_i) + \|x_{\text{new}} - x_i\|_2.$$

The selected parent becomes the parent pointer of x_{new}, and

$$g(x_{\text{new}}) = g(x_{\text{parent}}) + \|x_{\text{new}} - x_{\text{parent}}\|_2.$$

Therefore, the new node is attached to the nearby node that gives the cheapest path from the start, not necessarily the closest node geometrically.

c) Rewiring Step: After x_{new} is added, the planner checks whether it can improve the costs of nearby nodes. For each $x_i \in \mathcal{N}(x_{\text{new}})$, it computes

$$g(x_{\text{new}}) + \|x_i - x_{\text{new}}\|_2.$$

If

$$g(x_{\text{new}}) + \|x_i - x_{\text{new}}\|_2 < g(x_i),$$

and the edge from x_{new} to x_i is collision-free, then x_i's parent is changed to x_{new}, and its cost-to-come is updated. This is the key difference from vanilla RRT: RRT* not only grows the tree, but also locally repairs it when cheaper connections are found. Repeated rewiring gradually reduces cost-to-come values as samples are added, which is the mechanism behind RRT*'s asymptotic optimality.

Algorithm 2 Static RRT* Planner

Initialize $V = \{x_s\}$, with $g(x_s) = 0$
Store x_s in `nodes` and `coordinates`
Build KDTree over `coordinates`
for $i = 1$ to $N_{\max}$ **do**
  Sample x_{rand} using goal bias p_{goal}
  Find nearest node x_{near} using KDTree
  Extend from x_{near} toward x_{rand} by at most ϵ to get x_{new}
  Initialize $g(x_{\text{new}}) = \infty$
  if edge $(x_{\text{near}}, x_{\text{new}})$ is in collision **then**
    Reject x_{new} and continue
  end if
  Find nearby nodes $\mathcal{N}(x_{\text{new}})$ within radius r
  Set $x_{\text{parent}} = x_{\text{near}}$
  Set $g_{\min} = g(x_{\text{near}}) + \|x_{\text{new}} - x_{\text{near}}\|_2$
  for each $x_i \in \mathcal{N}(x_{\text{new}})$ **do**
    if edge (x_i, x_{new}) is collision-free **then**
      $g_{\text{candidate}} = g(x_i) + \|x_{\text{new}} - x_i\|_2$
      if $g_{\text{candidate}} < g_{\min}$ **then**
        $x_{\text{parent}} = x_i$
        $g_{\min} = g_{\text{candidate}}$
      end if
    end if
  end for
  Set parent of x_{new} to x_{parent}
  Set $g(x_{\text{new}}) = g_{\min}$
  Add x_{new} to `nodes` and `coordinates`
  Rebuild KDTree with updated `coordinates`
  for each $x_i \in \mathcal{N}(x_{\text{new}})$ **do**
    $g_{\text{rewire}} = g(x_{\text{new}}) + \|x_i - x_{\text{new}}\|_2$
    if $g_{\text{rewire}} < g(x_i)$ and edge (x_{new}, x_i) is collision-free **then**
      Set parent of x_i to x_{new}
      Update $g(x_i) = g_{\text{rewire}}$
    end if
  end for
  if x_{new} reaches x_g **then**
    Backtrack through parent pointers and return path
  end if
end for
Return failure if no goal node is found

The planner terminates when a sampled extension reaches the goal or when $N_{\max}$ is reached. If a goal node is found, the path is recovered by backtracking through parent pointers and reversing the sequence. The path length is the sum of Euclidean distances between consecutive path points.

Memory usage is $O(|V|)$, since each sampled node contributes one coordinate, one parent pointer, and one cost value.

The main computational costs are nearest-neighbor search, neighborhood search, collision checking, and rewiring. The KDTree reduces nearest-neighbor lookup from a linear scan over all nodes to efficient spatial queries, while collision checking has worst-case cost $O(m)$ per edge.

The planner is probabilistically complete rather than deterministically complete. If a feasible path exists, the probability of finding one approaches one as the number of samples increases. Under standard RRT* assumptions and with sufficiently many samples, the planner is asymptotically optimal. With a finite iteration limit, however, it is not guaranteed to return the exact shortest path.

I chose RRT* instead of vanilla A* or vanilla RRT because the problem is continuous and three-dimensional. A* would require discretizing the full 3-D configuration space into a grid, which can be expensive in memory and runtime. A coarse grid can miss narrow passages or produce low-quality paths. Vanilla RRT avoids the grid but does not optimize parent choices after nodes are added. RRT* keeps the continuous-space advantages of RRT while improving path quality through best-parent selection and rewiring.

C. Part 3: Fast Replanning for Dynamic Goals

For E6–E7, the start position and obstacle map remain fixed, but the target moves through eight goal positions,

$$x_{g,1}, x_{g,2}, \ldots, x_{g,8}.$$

For each revealed goal, the planner computes a path from the same start to the current goal before the target moves again. The planner does not precompute paths to future goals.

To enable fast replanning, I reused the RRT* tree across goal queries. On the first query, the planner initializes the tree from the start. On later queries, it reuses the stored nodes, parent pointers, costs, coordinates, and KDTree, then continues growing the same tree toward the new goal. In the implementation, this reused state is stored as

`self.nodes`, `self.coordinates`, `self.tree`.

This is valid because the obstacle map is static, so previously sampled collision-free nodes and edges remain valid for later goals.

This dynamic extension is similar in spirit to LPA* or D* because information from previous planning queries is reused. However, the reused structure is not a discrete graph with updated vertex values; it is a continuous RRT* tree containing sampled configurations, parent pointers, g-values, and a nearest-neighbor structure.

The benefit is that later goals can connect to regions of free space already explored for earlier goals, reducing repeated exploration compared with restarting RRT* from only the start node. The tradeoff is that memory usage grows as samples accumulate, and the KDTree must be rebuilt or updated when new samples are added. Since only the goal changes, tree reuse is appropriate for fast replanning.

Algorithm 3 Dynamic Replanning with Reused RRT* Tree

```
Initialize planner with empty stored tree
for each revealed goal x_{g,k}, k = 1, ..., 8 do
    if stored tree is empty then
        Initialize tree from start x_s, with g(x_s) = 0
    else
        Reuse stored nodes, coordinates, and KDTree
    end if
    Run RRT* toward current goal x_{g,k}
    Backtrack from the reached goal node to obtain the current path
    Store the updated tree for the next goal query
    Record path length, runtime, iterations, node count, and timing success
end for
```

IV. Results

The planner was evaluated on static-goal environments E1–E5 and dynamic-goal environments E6–E7. Since RRT* is sampling-based, runtime, explored nodes, and path length can vary between runs. The reported values correspond to one representative run for each parameter setting. The general trend was consistent: smaller steering distances usually improved path quality but required more computation, while larger steering distances reduced runtime but could produce less refined paths.

For the static environments, path quality is reported using path length, computational effort using explored nodes and iterations, and runtime. The parameters varied were the steering distance ϵ, goal bias probability p_{goal}, and maximum iteration count $N_{\max}$.

Across the static-goal experiments, $\epsilon = 0.5$, $p_{\text{goal}} = 0.1$, and $N_{\max} = 100000$ was selected as the best balanced setting for final visualizations. This setting gave finer path refinement than $\epsilon = 1.0$ while still solving the difficult environments with a sufficiently large iteration budget. In contrast, $\epsilon = 1.0$ was often faster, especially in Maze and Monza, but used coarser extensions. Thus, $\epsilon = 1.0$ was better for runtime, while $\epsilon = 0.5$ better balanced runtime and path quality.

A. Static-Goal Environments

1) E1: Flappy Bird: The Flappy Bird map contains several vertical obstacle walls, so the planner must move through gaps while progressing toward the goal. Table I shows the parameter sweep. All tested settings found collision-free paths.

Smaller steering distances produced shorter paths, especially $\epsilon = 0.25$, but required more nodes and runtime. Larger steering distances solved the map faster but generally produced longer paths. For this map, $\epsilon = 0.5$ gave a good balance between efficiency and solution quality.

2) E2: Maze: The Maze environment was significantly harder because the planner must navigate through a long constrained passage. Table II shows that many settings failed before reaching the iteration limit, which is expected for sampling-based planning in narrow passages.

TABLE I
FLAPPY BIRD PARAMETER SWEEP FOR RRT*.

ϵ	p_g	$N_{\max}$	Nodes	Iter.	Succ.	Len./Time
0.25	0.05	10000	1212	2217	T	25.64 / 1.84
0.25	0.05	50000	1166	2034	T	25.99 / 1.70
0.25	0.05	100000	1220	2257	T	26.08 / 1.84
0.25	0.10	10000	898	1789	T	26.31 / 1.21
0.25	0.10	50000	1045	2164	T	26.43 / 1.52
0.25	0.10	100000	983	1729	T	26.03 / 1.30
0.25	0.20	10000	1205	2942	T	27.43 / 1.85
0.25	0.20	50000	940	2187	T	25.75 / 1.33
0.25	0.20	100000	983	2384	T	26.53 / 1.47
0.50	0.05	10000	706	1243	T	27.66 / 0.72
0.50	0.05	50000	511	919	T	27.07 / 0.46
0.50	0.05	100000	522	945	T	26.37 / 0.50
0.50	0.10	10000	767	1317	T	26.98 / 0.75
0.50	0.10	50000	374	803	T	26.90 / 0.31
0.50	0.10	100000	693	1380	T	26.37 / 0.76
0.50	0.20	10000	547	1216	T	26.92 / 0.52
0.50	0.20	50000	720	1571	T	26.58 / 0.78
0.50	0.20	100000	535	1209	T	26.88 / 0.49
1.00	0.05	10000	239	440	T	27.68 / 0.14
1.00	0.05	50000	294	575	T	28.31 / 0.21
1.00	0.05	100000	412	632	T	27.70 / 0.28
1.00	0.10	10000	355	681	T	29.00 / 0.22
1.00	0.10	50000	231	514	T	28.53 / 0.14
1.00	0.10	100000	284	717	T	27.83 / 0.20
1.00	0.20	10000	182	387	T	28.34 / 0.10
1.00	0.20	50000	177	483	T	27.19 / 0.09
1.00	0.20	100000	223	461	T	27.20 / 0.14

TABLE II
MAZE PARAMETER SWEEP FOR RRT*.

ϵ	p_g	$N_{\max}$	Nodes	Iter.	Succ.	Len./Time
0.25	0.05	10000	560	10000	F	– / 6.64
0.25	0.05	50000	6009	50000	F	– / 85.10
0.25	0.05	100000	18312	100000	F	– / 270.75
0.25	0.10	10000	592	10000	F	– / 6.77
0.25	0.10	50000	4986	50000	F	– / 74.31
0.25	0.10	100000	16695	100000	F	– / 245.40
0.25	0.20	10000	454	10000	F	– / 4.80
0.25	0.20	50000	6118	50000	F	– / 71.54
0.25	0.20	100000	15428	100000	F	– / 206.19
0.50	0.05	10000	910	10000	F	– / 8.23
0.50	0.05	50000	8574	50000	F	– / 95.13
0.50	0.05	100000	12634	64123	T	73.06 / 139.48
0.50	0.10	10000	856	10000	F	– / 6.98
0.50	0.10	50000	9344	50000	F	– / 97.69
0.50	0.10	100000	12358	65477	T	73.49 / 134.80
0.50	0.20	10000	526	10000	F	– / 5.59
0.50	0.20	50000	9133	47183	T	74.28 / 78.33
0.50	0.20	100000	12113	67679	T	73.23 / 127.90
1.00	0.05	10000	1277	10000	F	– / 9.10
1.00	0.05	50000	6636	26435	T	74.15 / 48.82
1.00	0.05	100000	7049	31720	T	74.63 / 59.95
1.00	0.10	10000	1044	10000	F	– / 7.15
1.00	0.10	50000	8326	42216	T	73.85 / 76.40
1.00	0.10	100000	8277	40960	T	73.69 / 72.97
1.00	0.20	10000	775	10000	F	– / 5.98
1.00	0.20	50000	6658	37335	T	73.85 / 55.27
1.00	0.20	100000	8322	44783	T	74.72 / 73.66

Increasing $N_{\max}$ was necessary for Maze. Larger steering distances, especially $\epsilon = 1.0$, helped the tree move through the maze more efficiently and reduced runtime compared to $\epsilon = 0.5$. Successful path lengths were consistently around 73–75, indicating that once the passage was found, path quality was similar across successful runs.

3) E3: Monza: The Monza environment was one of the most difficult static-goal maps because its obstacle layout creates a long constrained route. Table III shows that successful runs mainly occurred when $N_{\max} = 100000$.

For $\epsilon = 0.25$, none of the tested settings found a path. For $\epsilon = 0.5$ and $\epsilon = 1.0$, the planner succeeded only in some high-iteration runs. Successful path lengths were all close to 73, but runtime was high because many nodes were needed before reaching the goal. The fastest successful setting was $\epsilon = 1.0$, $p_{\text{goal}} = 0.1$, and $N_{\max} = 100000$.

4) E4: Single Cube: The Single Cube environment was the easiest static-goal map because it contains only one main obstacle block. As shown in Table IV, every tested parameter setting found a collision-free path.

TABLE III
MONZA PARAMETER SWEEP FOR RRT*.

ϵ	p_g	$N_{\max}$	Nodes	Iter.	Succ.	Len./Time
0.25	0.05	10000	2890	10000	F	– / 3.82
0.25	0.05	50000	17983	50000	F	– / 58.15
0.25	0.05	100000	51218	100000	F	– / 349.64
0.25	0.10	10000	2748	10000	F	– / 3.70
0.25	0.10	50000	17060	50000	F	– / 53.16
0.25	0.10	100000	43227	100000	F	– / 256.60
0.25	0.20	10000	2439	10000	F	– / 2.96
0.25	0.20	50000	14880	50000	F	– / 42.21
0.25	0.20	100000	37619	100000	F	– / 196.08
0.50	0.05	10000	2876	10000	F	– / 3.55
0.50	0.05	50000	23994	50000	F	– / 84.03
0.50	0.05	100000	43734	70694	T	73.24 / 242.57
0.50	0.10	10000	2888	10000	F	– / 3.56
0.50	0.10	50000	20686	50000	F	– / 66.60
0.50	0.10	100000	53448	91482	T	73.24 / 358.15
0.50	0.20	10000	2383	10000	F	– / 2.77
0.50	0.20	50000	18175	50000	F	– / 53.26
0.50	0.20	100000	51282	100000	F	– / 331.74
1.00	0.05	10000	3218	10000	F	– / 3.95
1.00	0.05	50000	27495	50000	F	– / 103.59
1.00	0.05	100000	38682	67823	T	73.32 / 193.06
1.00	0.10	10000	3002	10000	F	– / 3.51
1.00	0.10	50000	25371	50000	F	– / 89.75
1.00	0.10	100000	32776	58442	T	73.32 / 140.36
1.00	0.20	10000	2691	10000	F	– / 3.03
1.00	0.20	50000	20712	50000	F	– / 64.33
1.00	0.20	100000	39719	80528	T	73.48 / 202.91

TABLE IV
SINGLE CUBE PARAMETER SWEEP FOR RRT*.

ϵ	p_g	$N_{\max}$	Nodes	Iter.	Succ.	Len./Time
0.25	0.05	10000	301	300	T	7.94 / 0.219
0.25	0.05	50000	252	251	T	7.92 / 0.178
0.25	0.05	100000	263	262	T	7.90 / 0.195
0.25	0.10	10000	190	189	T	7.88 / 0.111
0.25	0.10	50000	288	292	T	7.88 / 0.204
0.25	0.10	100000	281	293	T	7.97 / 0.194
0.25	0.20	10000	139	138	T	7.94 / 0.067
0.25	0.20	50000	140	142	T	7.90 / 0.069
0.25	0.20	100000	126	125	T	7.90 / 0.058
0.50	0.05	10000	169	168	T	7.95 / 0.079
0.50	0.05	50000	116	116	T	7.99 / 0.042
0.50	0.05	100000	94	93	T	7.93 / 0.034
0.50	0.10	10000	129	128	T	8.45 / 0.054
0.50	0.10	50000	122	121	T	7.89 / 0.051
0.50	0.10	100000	137	139	T	7.95 / 0.061
0.50	0.20	10000	64	63	T	7.96 / 0.017
0.50	0.20	50000	87	86	T	7.91 / 0.029
0.50	0.20	100000	99	98	T	7.89 / 0.035
1.00	0.05	10000	50	50	T	8.27 / 0.010
1.00	0.05	50000	78	78	T	8.02 / 0.022
1.00	0.05	100000	116	115	T	8.02 / 0.037
1.00	0.10	10000	35	34	T	8.08 / 0.006
1.00	0.10	50000	82	84	T	8.13 / 0.023
1.00	0.10	100000	43	42	T	7.89 / 0.008
1.00	0.20	10000	34	33	T	7.95 / 0.005
1.00	0.20	50000	31	30	T	7.95 / 0.005
1.00	0.20	100000	51	50	T	7.95 / 0.011

Compared with Maze and Monza, Single Cube required far fewer nodes and much less runtime. Increasing p_{goal} generally reduced the node count because the direct goal direction was not repeatedly blocked by a complex obstacle layout. The fastest runs occurred with $\epsilon = 1.0$, while smaller steering distances produced similar path quality with slightly more computation.

5) E5: Tower: The Tower environment requires the planner to move from a low starting position to a much higher goal while avoiding vertical obstacle structures. Unlike Maze and Monza, all tested settings found collision-free paths.

Tower shows a clear quality–runtime tradeoff. Smaller steering distances produced shorter paths but required more computation, while $\epsilon = 1.0$ gave the fastest solutions. For this environment, $\epsilon = 0.5$ was a practical compromise because it produced paths shorter than most $\epsilon = 1.0$ runs while remaining much faster than $\epsilon = 0.25$.

TABLE V
TOWER PARAMETER SWEEP FOR RRT*.

ϵ	p_g	$N_{\max}$	Nodes	Iter.	Succ.	Len./Time
0.25	0.05	10000	2309	6136	T	28.88 / 13.95
0.25	0.05	50000	2804	6885	T	28.71 / 17.57
0.25	0.05	100000	1658	4270	T	29.12 / 9.00
0.25	0.10	10000	2325	6049	T	28.82 / 14.08
0.25	0.10	50000	1569	4070	T	28.29 / 8.50
0.25	0.10	100000	3694	7608	T	28.48 / 24.33
0.25	0.20	10000	1986	5814	T	29.03 / 11.87
0.25	0.20	50000	1930	6545	T	28.34 / 11.66
0.25	0.20	100000	1831	5426	T	29.30 / 9.48
0.50	0.05	10000	1626	3847	T	29.26 / 7.89
0.50	0.05	50000	1471	2905	T	29.83 / 5.60
0.50	0.05	100000	1099	2240	T	29.65 / 4.01
0.50	0.10	10000	1374	3309	T	29.86 / 5.79
0.50	0.10	50000	662	1849	T	29.39 / 2.16
0.50	0.10	100000	893	2047	T	29.58 / 3.05
0.50	0.20	10000	919	2359	T	29.40 / 3.24
0.50	0.20	50000	1577	4377	T	28.87 / 6.39
0.50	0.20	100000	2232	5988	T	29.16 / 11.27
1.00	0.05	10000	553	1317	T	29.55 / 1.33
1.00	0.05	50000	603	1424	T	31.12 / 1.65
1.00	0.05	100000	448	1312	T	31.16 / 1.15
1.00	0.10	10000	513	1291	T	30.45 / 1.28
1.00	0.10	50000	651	1592	T	30.36 / 1.78
1.00	0.10	100000	1006	2657	T	30.58 / 3.76
1.00	0.20	10000	933	2095	T	30.51 / 2.57
1.00	0.20	50000	509	1769	T	30.58 / 1.38
1.00	0.20	100000	790	2331	T	30.91 / 2.47

B. Dynamic-Goal Environments

The dynamic-goal environments E6–E7 require repeated planning from the same start to eight moving target positions. The same RRT* planner was used for each target, but the tree was reused across queries. Since the obstacle map is fixed, previously sampled collision-free nodes and edges remain valid. Therefore, the node count in the dynamic tables represents cumulative tree size after each goal. In the tables, Path/Time reports geometric path success and timing success, respectively.

1) E6: Window: The Window environment required the planner to reach eight target locations arranged around the window structure. Table VI shows per-goal replanning performance using $\epsilon = 0.5$, $p_{\text{goal}} = 0.1$, and $N_{\max} = 100000$. The planner found collision-free paths to all eight goals, but goal 3 exceeded the tested target stay time in the representative run, giving timing success of $7/8$.

TABLE VI
E6 WINDOW PER-GOAL DYNAMIC REPLANNING RESULTS.

Goal	Nodes	Iter.	Path/Time	Stay	Length	Run/Cum.
1	122	200	T/T	3.00	19.79	0.12/0.12
2	159	43	T/T	1.00	20.47	0.05/0.18
3	1394	1421	T/F	1.00	20.06	2.08/2.26
4	1398	4	T/T	1.00	19.30	0.01/2.26
5	1405	8	T/T	1.00	18.42	0.01/2.27
6	1408	3	T/T	1.00	17.23	0.00/2.28
7	1417	9	T/T	1.00	17.27	0.02/2.30
8	1418	1	T/T	1.00	18.35	0.00/2.30

Tree reuse was especially helpful after goal 3. Once the tree had expanded enough to reach that region, goals 4–8 required only 1–9 additional iterations. This supports the idea that the reused RRT* tree acts like a roadmap of the environment.

Table VII shows that path feasibility was not the limiting factor; all timing pairs produced valid paths to all goals. Timing failures occurred only when the planner exceeded the allowed target stay time. The best tested timing pair was $t_1 = 1.0$, $t_2 = 0.5$, which achieved full success with total target period

$$t_1 + 7t_2 = 1.0 + 7(0.5) = 4.5 \text{ s}.$$

TABLE VII
E6 WINDOW TIMING SWEEP FOR DYNAMIC TARGETS.

t_1	t_2	Path Succ.	Time Succ.	Overall Succ.	Total Time
0.10	0.05	8/8	5/8	5/8	2.93
0.25	0.10	8/8	7/8	7/8	1.07
0.50	0.25	8/8	7/8	7/8	0.79
1.00	0.50	8/8	8/8	8/8	0.92
3.00	1.00	8/8	8/8	8/8	0.68

2) E7: Room: The Room environment also contains eight moving target positions, but the target locations are distributed throughout the room. Table VIII shows per-goal replanning results using $\epsilon = 0.5$, $p_{\text{goal}} = 0.1$, and $N_{\max} = 100000$. The planner found valid paths to all goals, but the smallest tested timing values caused failures for goals 3–5 in the representative run.

TABLE VIII
E7 ROOM PER-GOAL DYNAMIC REPLANNING RESULTS.

Goal	Nodes	Iter.	Path/Time	Stay	Length	Run/Cum.
1	32	58	T/T	0.30	4.56	0.03/0.03
2	46	29	T/T	0.10	8.78	0.02/0.05
3	109	148	T/F	0.10	12.85	0.13/0.18
4	175	102	T/F	0.10	4.39	0.15/0.33
5	271	139	T/F	0.10	17.18	0.25/0.57
6	279	13	T/T	0.10	15.88	0.02/0.60
7	282	4	T/T	0.10	15.79	0.01/0.60
8	299	22	T/T	0.10	17.19	0.04/0.65

The Room results show that the planner was geometrically successful for all goals, but goals 3–5 required more exploration and exceeded the tested target stay time. Once the tree became more developed, later goals required fewer iterations and were solved within the time limit.

TABLE IX
E7 ROOM TIMING SWEEP FOR DYNAMIC TARGETS.

t_1	t_2	Path Succ.	Time Succ.	Overall Succ.	Total Time
0.30	0.10	8/8	6/8	6/8	0.57
0.50	0.25	8/8	8/8	8/8	0.45
1.00	0.50	8/8	7/8	7/8	2.32
2.00	1.00	8/8	8/8	8/8	0.97
5.00	2.00	8/8	7/8	7/8	2.49

For Room, the best tested timing pair was $t_1 = 0.5$, $t_2 = 0.25$, which achieved full success with the smallest successful target stay times in the sweep. This corresponds to

$$t_1 + 7t_2 = 0.5 + 7(0.25) = 2.25 \text{ s}.$$

The timing sweep again shows the tradeoff between fast replanning and the time available before the target moves.

C. Visualization Results

The following figures show the final planned paths for the static-goal and dynamic-goal environments. For each map, the top image shows the 3-D path through the obstacle environment, and the bottom image shows the XY, XZ, and YZ plane projections. Apparent overlap in a 2-D projection does not necessarily imply collision because the path may be separated from the obstacle in the third dimension.

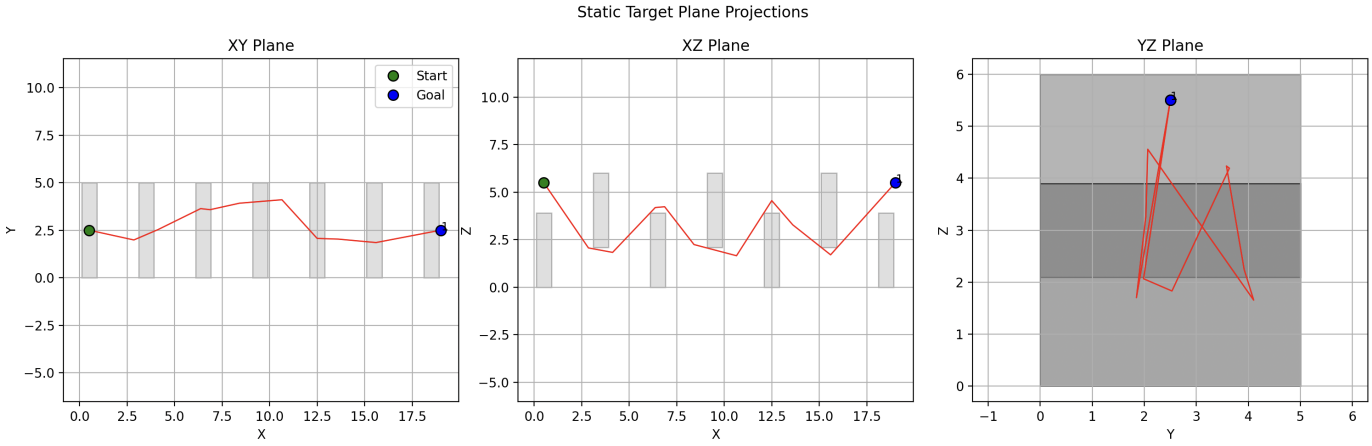


Fig. 1. E1 Flappy Bird environment. Top: 3-D RRT* path from start to goal. Bottom: XY, XZ, and YZ plane projections of the same path.

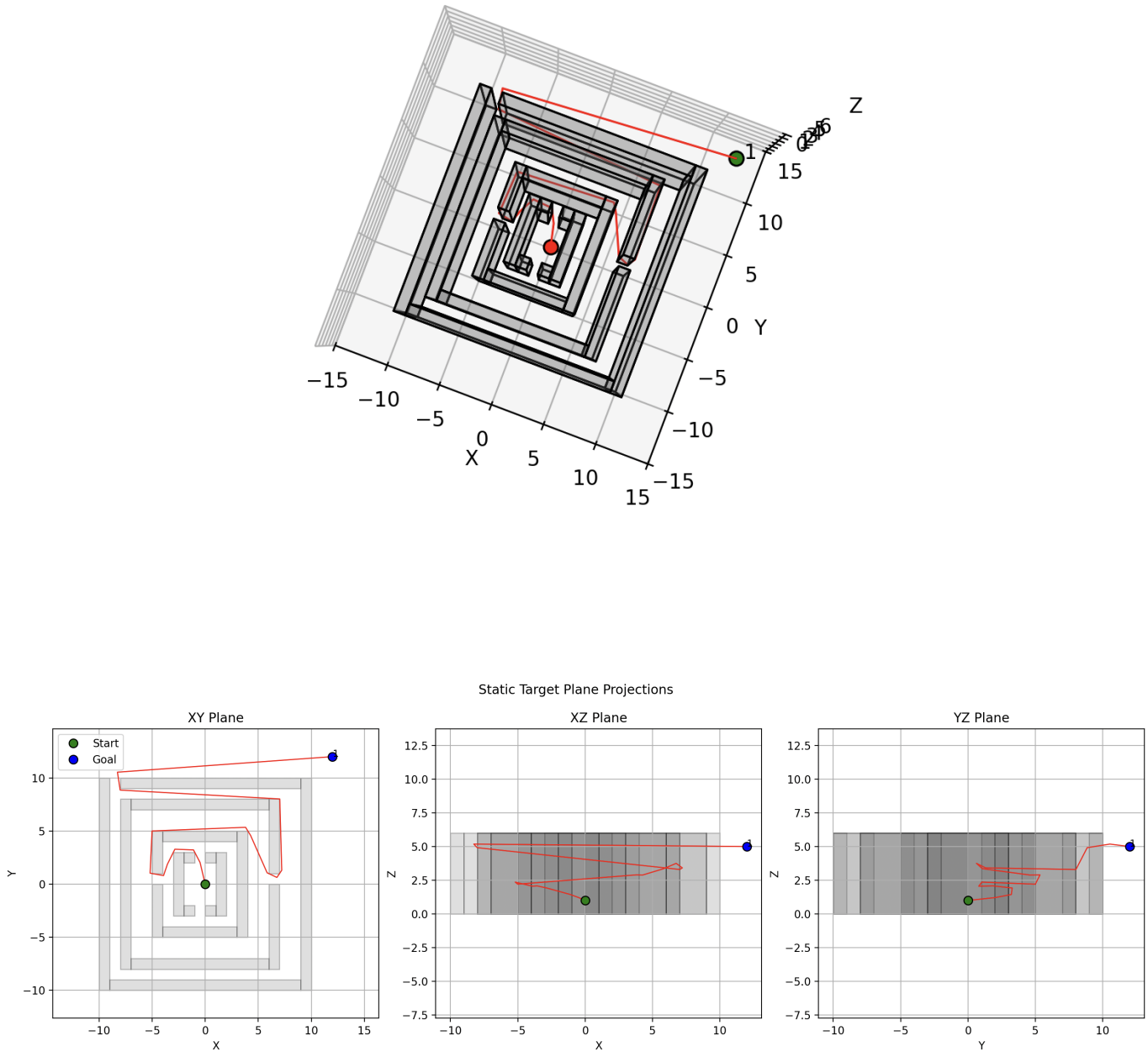


Fig. 2. E2 Maze environment. Top: 3-D RRT* path from start to goal. Bottom: XY, XZ, and YZ plane projections of the same path.

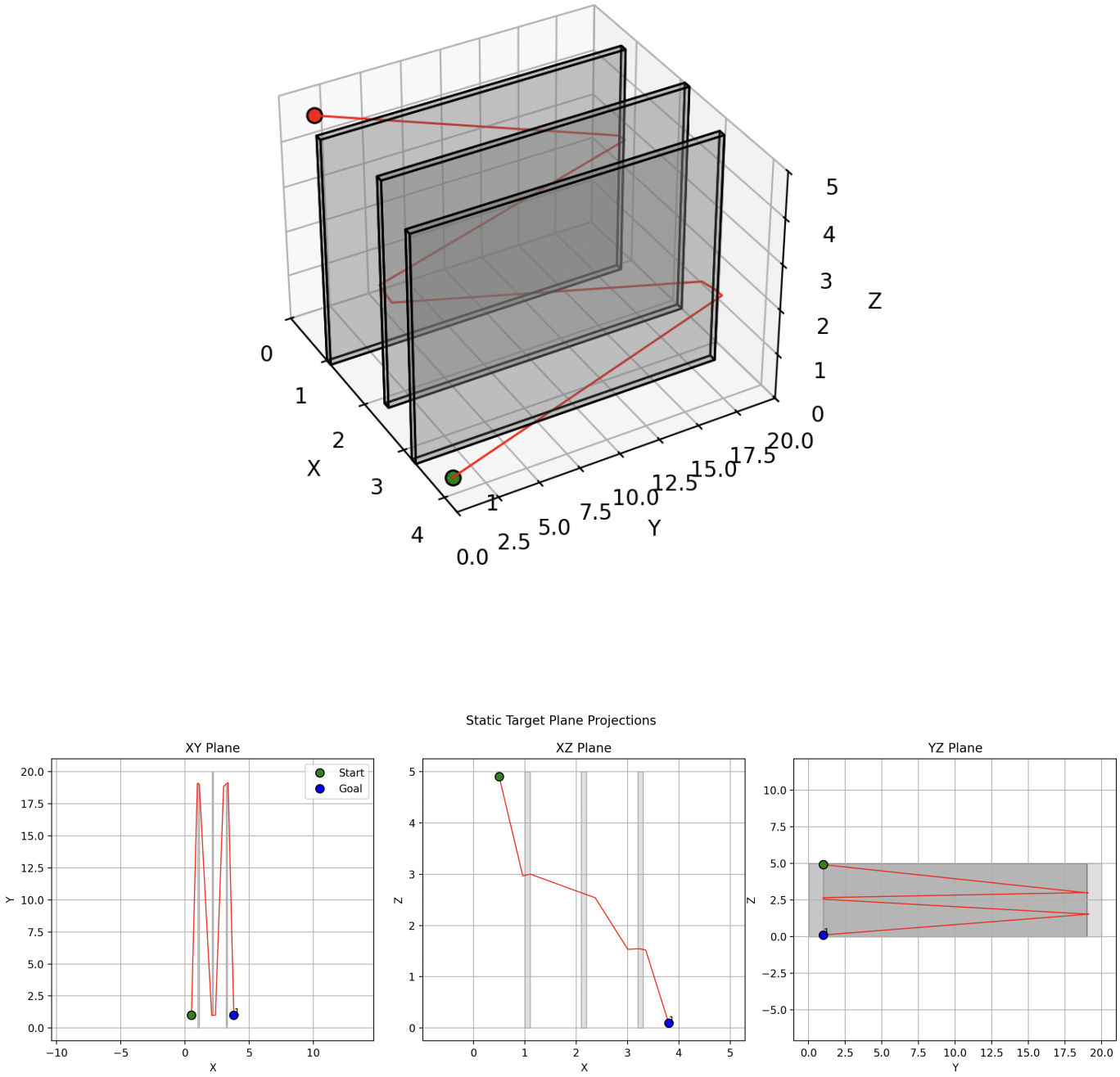
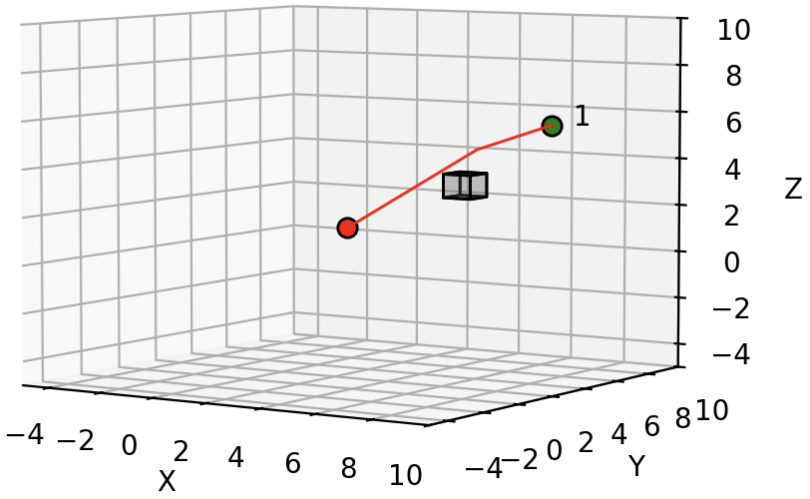


Fig. 3. E3 Monza environment. Top: 3-D RRT* path from start to goal. Bottom: XY, XZ, and YZ plane projections of the same path.



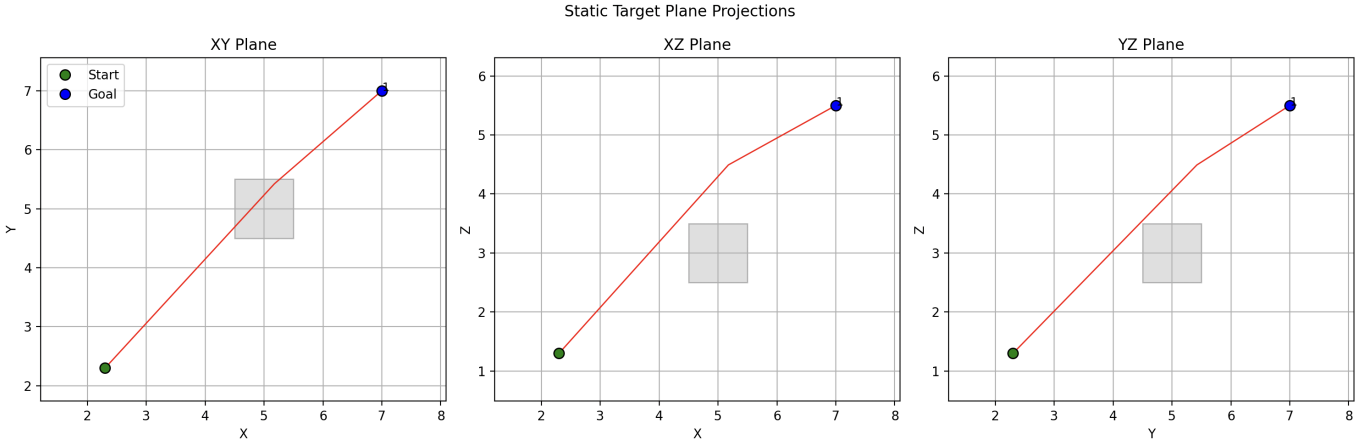


Fig. 4. E4 Single Cube environment. Top: 3-D RRT* path from start to goal. Bottom: XY, XZ, and YZ plane projections of the same path.

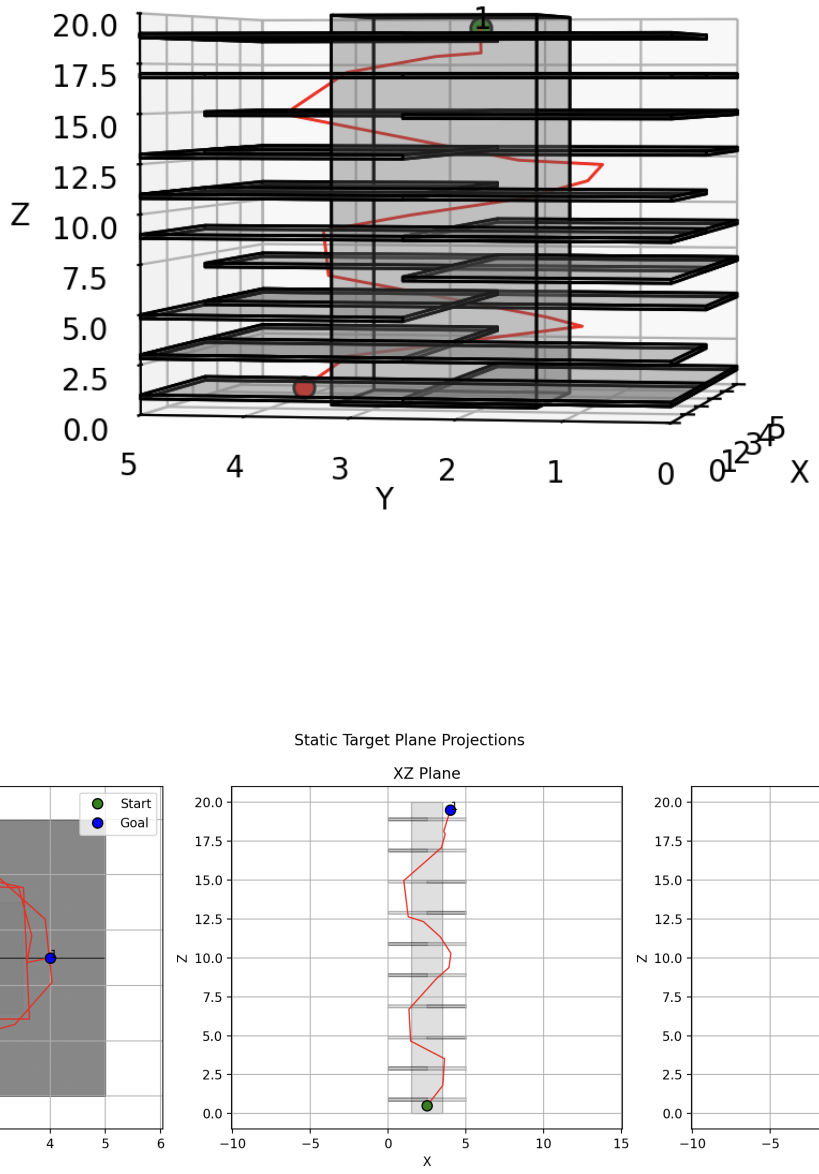


Fig. 5. E5 Tower environment. Top: 3-D RRT* path from start to goal. Bottom: XY, XZ, and YZ plane projections of the same path.

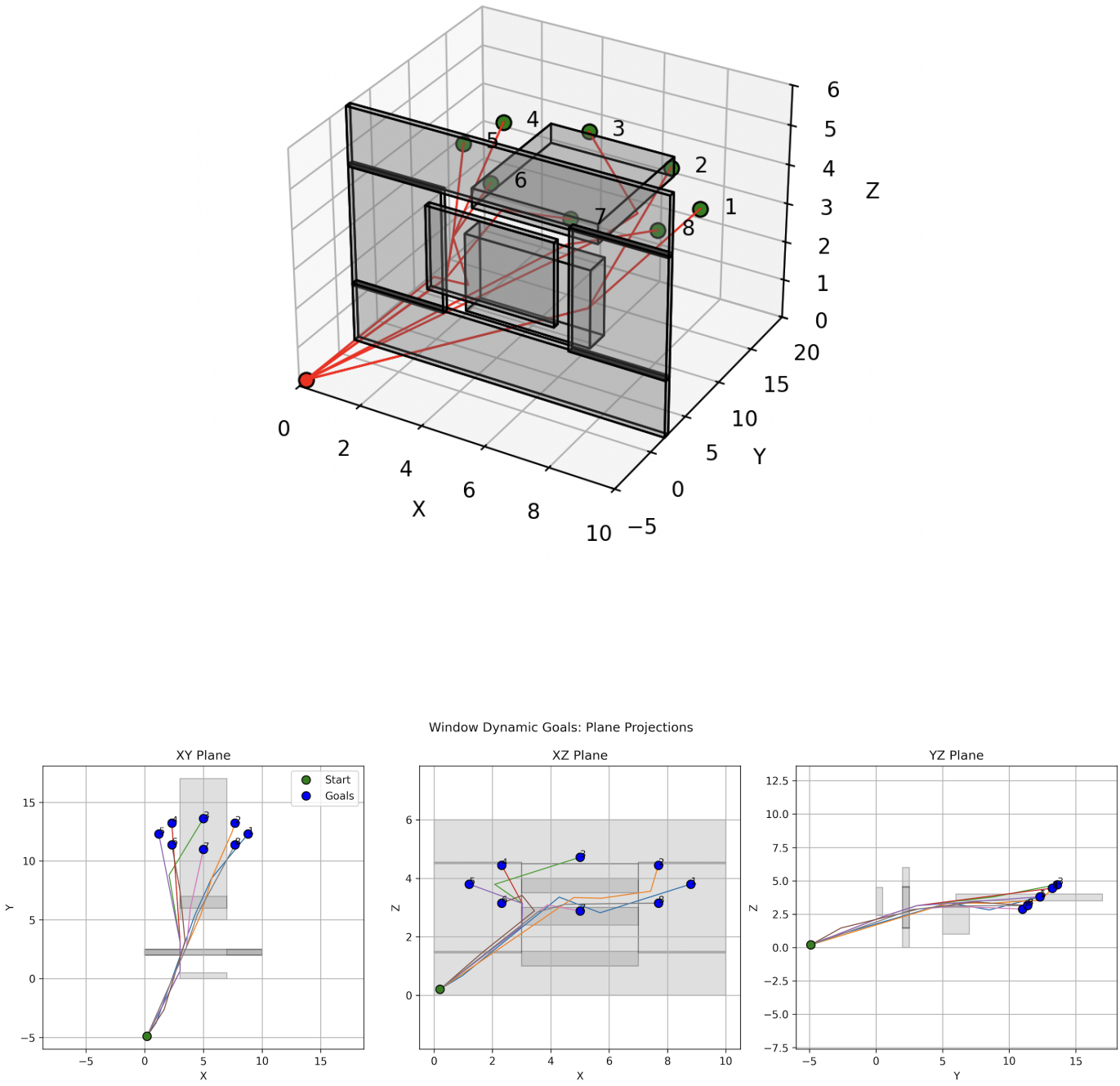


Fig. 6. E6 Window dynamic-goal results. Top: 3-D paths from the same start to the eight revealed goals. Bottom: XY, XZ, and YZ plane projections with obstacle blocks.

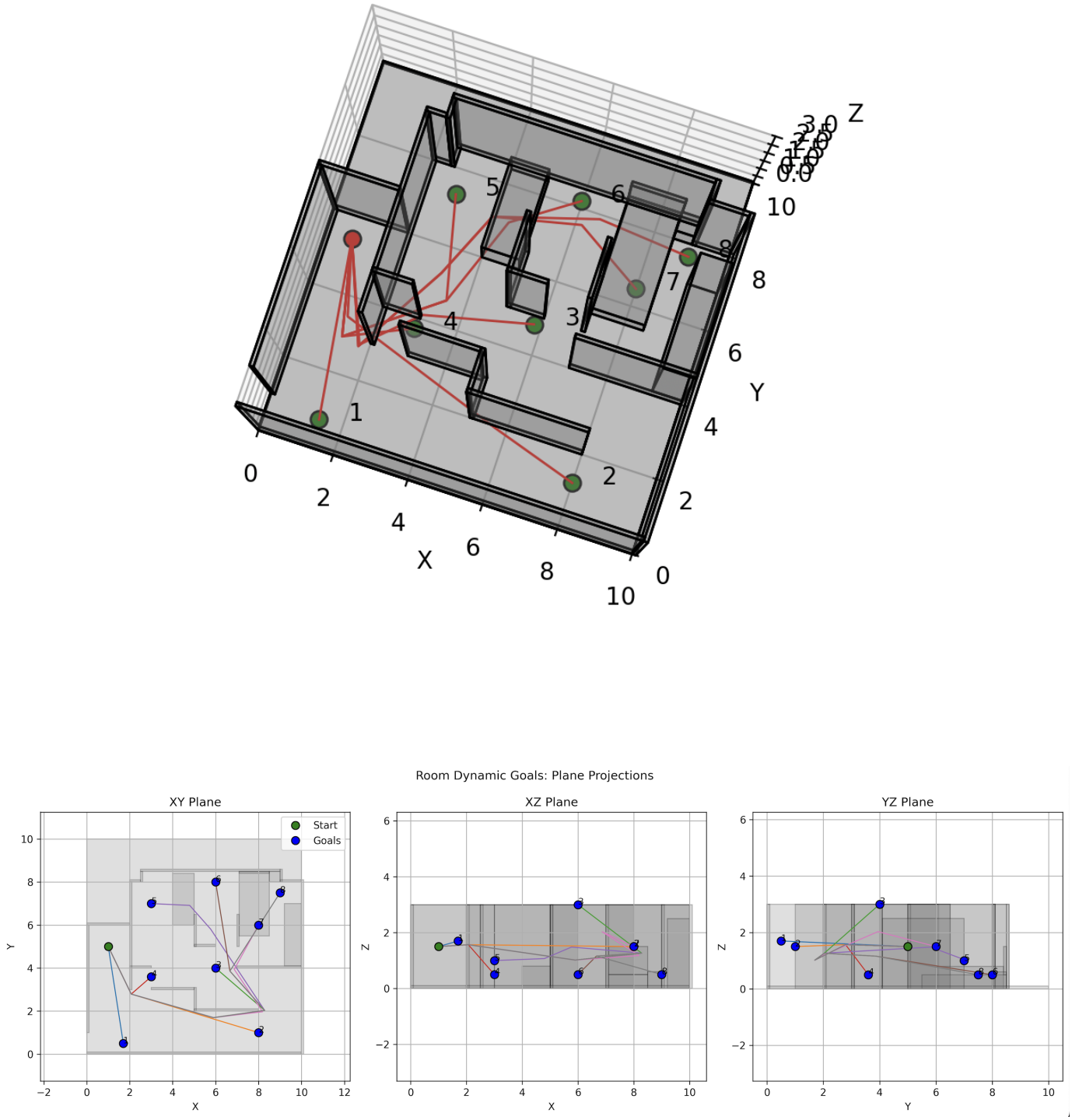


Fig. 7. E7 Room dynamic-goal results. Top: 3-D paths from the same start to the eight revealed goals. Bottom: XY, XZ, and YZ plane projections with obstacle blocks.